



Authentication & Authorization

Securing bank_api · Extension 6

From "anyone can withdraw from any account" to "only you can move your money"

Password hashing

JWT

FastAPI dependencies

RBAC

Ownership (BOLA)

WHY THIS MODULE EXISTS

Right now, bank_api trusts everyone

```
terminal
# Extension 5 - no login, no token, no questions asked
$ curl -X POST localhost:8000/api/v1/accounts/1/withdraw \
  -H "Content-Type: application/json" \
  -d '{"amount": 5000}'

HTTP/1.1 200 OK
{"id": 1, "owner_name": "Asha", "balance": 0.0}

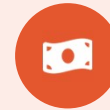
# ...and Asha never even knew who did it.
```

Every caller is anonymous. So far we have built a correct API - but not a safe one.



Read any balance

GET /api/v1/accounts/{id} works for any id



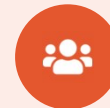
Move anyone's money

withdraw, transfer, schedule payments



Self-certify KYC

PUT ../kyc-status
{"kyc_compliant": true}



List every customer

GET /api/v1/accounts/ returns all accounts

LEARNING OUTCOMES

What you will be able to do by the end

01

Explain

the difference between authentication and authorization, and map each failure to 401, 403 or 404

02

Store

passwords safely with a salted, slow hash (Argon2id) and follow current NIST password rules

03

Issue & verify

short-lived JWTs - and know exactly which checks stop forged, expired or tampered tokens

04

Protect

FastAPI routes with dependencies, at route level and at router level

05

Enforce

roles (RBAC) AND ownership - the object-level check that OWASP ranks as API risk #1

06

Apply it

to bank_api in a 6-task hands-on lab with 57 new acceptance tests

CORE CONCEPTS

Authentication vs Authorization



Authentication (AuthN)

Who are you?

Bank branch	You show your ID card at the counter.
In bank_api	Username + password -> POST /auth/token -> JWT
Checked by	<code>get_current_user()</code>
Fails with	401 Unauthorized ("we don't know you")



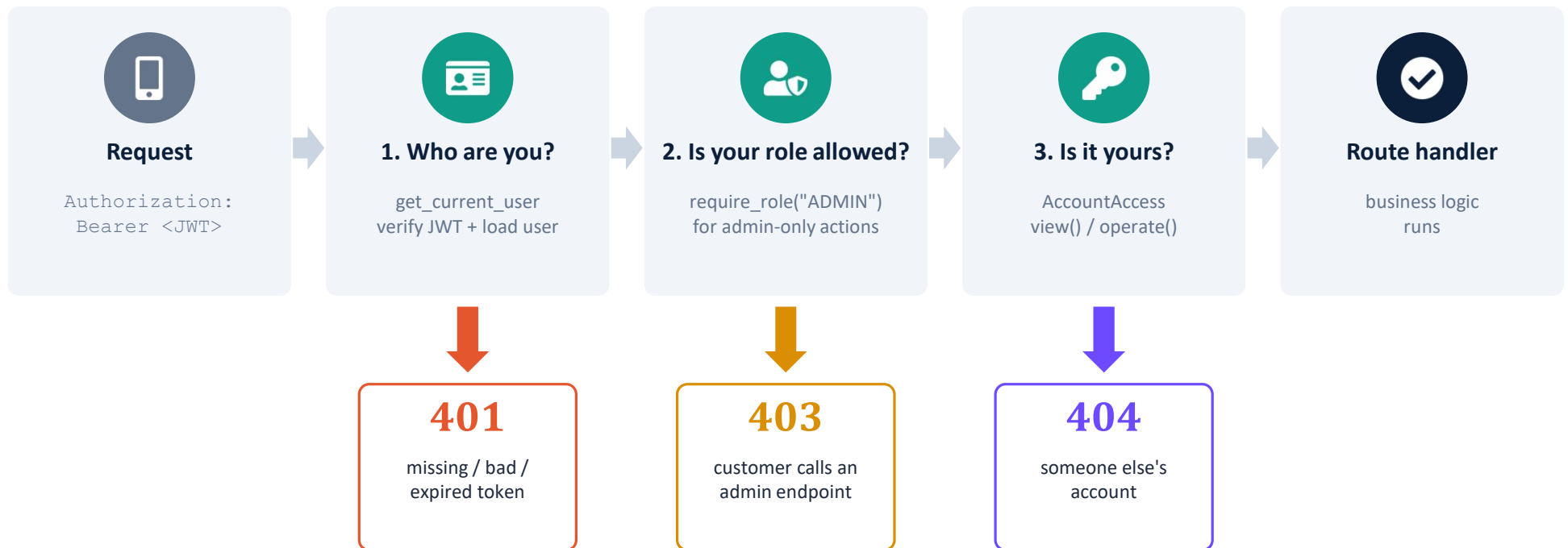
Authorization (AuthZ)

What are you allowed to do?

Bank branch	The teller lets you withdraw from YOUR account only; the vault is staff-only.
In bank_api	Customer: own accounts. Admin: certify KYC, view all.
Checked by	<code>require_role()</code> + <code>AccountAccess</code>
Fails with	403 Forbidden or 404 Not Found (hide others' data)

Order matters: AuthN always runs first - you cannot decide what someone may do until you know who they are.

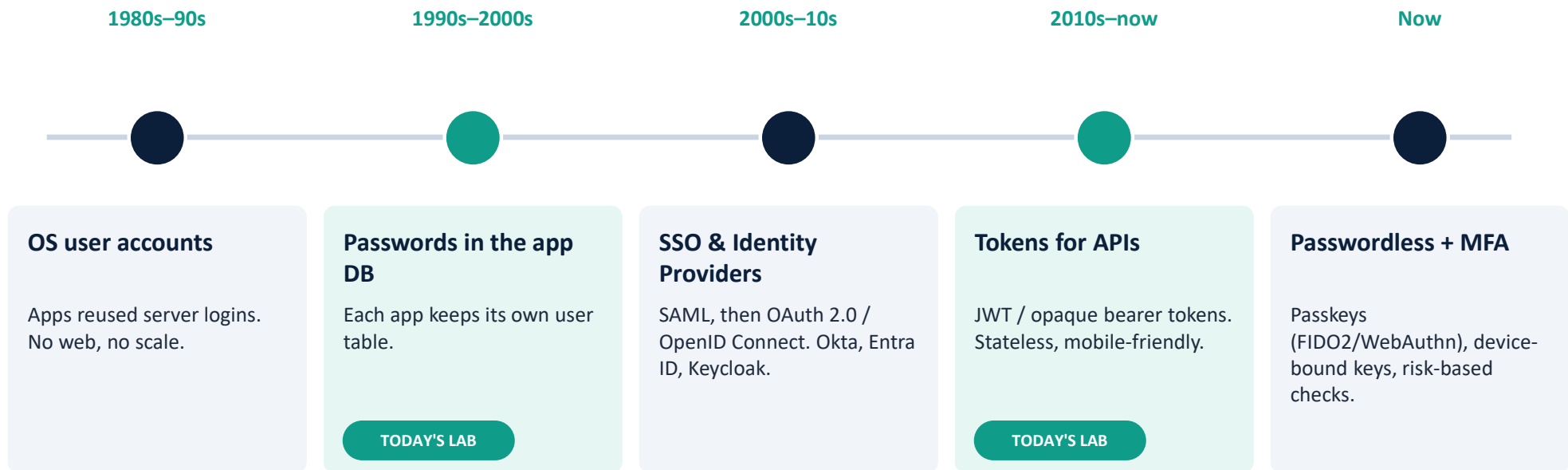
Every protected request answers three questions



In FastAPI each question is a dependency. They chain: require_role and AccountAccess both depend on get_current_user, so the token is decoded once per request.

CONTEXT

How authentication evolved



Real banks usually delegate login to an Identity Provider and only VERIFY tokens in their APIs. The code shape you build today - `get_current_user`, `require_role`, `ownership checks` - stays exactly the same.

TASK 1 FOUNDATION

Storing passwords: the only acceptable way



Plain text

One DB leak = every customer compromised (and their other sites).



Fast hash (MD5, SHA-256)

GPUs test billions per second; identical passwords give identical hashes.



Salted, slow hash (Argon2id, bcrypt, scrypt)

Each guess costs real memory and time; same password -> different hash.

What our API actually stores (pwdlib, Argon2id):

```
$argon2id$v=19$m=65536,t=3,p=4$NphCAR8d6NiU...$G1c6PSb6...
```

algorithm · cost: 64 MiB, 3 passes, 4 lanes · salt · hash



NIST SP 800-63B-4 (2025) password rules

- **Length:** ≥ 15 chars if the password is the only factor (≥ 8 with MFA)
- **Allow long:** accept at least 64 chars; passphrases welcome
- **No composition rules:** no forced symbols/uppercase mixes
- **No forced rotation:** change only on evidence of compromise
- **Blocklist:** reject known-breached and common passwords

Password vs Passkey

	Password	Passkey (FIDO2 / WebAuthn)
What it is	A shared secret the user remembers	A key pair; private key never leaves the device
Server stores	A salted slow hash of the secret	Only the PUBLIC key - useless to a thief
Phishing	Can be typed into a fake site	Bound to the real domain - fake sites get nothing
After a DB breach	Hashes can be cracked offline	Nothing to crack
User action	Remember and type it	Face / fingerprint / device PIN
Adoption (2026)	Universal	Growing fast; most banks offer it alongside passwords

Why we still build passwords today: passkeys replace the LOGIN step only. After login, your API still issues a token and still needs every AuthZ check in this module.

AUTHENTICATION SCHEMES

Choosing how callers prove who they are

Scheme	How it works	Server state	Revoke instantly?	Best for
Session cookie	Server stores session; browser sends its id in an HttpOnly cookie	Yes (session store)	Yes - delete session	Server-rendered web apps
JWT access token	Signed, self-contained claims in Authorization: Bearer	No	No - wait for exp (keep it short)	APIs, mobile & SPA clients, microservices <- bank_api
Opaque token	Random string; API asks the auth server (introspection)	Yes	Yes	High-risk APIs needing instant revocation
API key	Long random secret per client application	Yes (key table)	Yes - rotate key	Server-to-server integrations (not end users)
mTLS	Client proves identity with a certificate at TLS level	PKI	Yes - revoke cert	Bank-to-bank, open-banking regulators

Also worth knowing: PASETO - a "secure by default" alternative to JWT with no algorithm choice to get wrong. Same ideas, fewer foot-guns; far smaller ecosystem.

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiI3Iiwicm9sZSI6IktNVU1RPTUVSIIiwiaXNzIjoiYmFua19hcGkiLCJpYXQiOiJlE3ODk4ODg4OTYsImV4cCI6MTc4OTg4OTc5Nn0.hDfq2SF8rb-MjwmkRPovoX9Qvv6krjn5Zia6G_EDsF4

Header

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

How it was signed. The server must NOT trust this blindly - it pins the algorithm itself.

Payload (claims)

```
{
  "sub": "7",           # user id
  "role": "CUSTOMER",
  "iss": "bank_api",
  "iat": 1789888896,    # issued
  "exp": 17898889796    # +15 min
}
```

Base64Url-encoded, NOT encrypted. Anyone holding the token can read it.

Signature

```
HMAC-SHA256(  
    header + "." + payload,  
    JWT_SECRET_KEY  
)
```

Proves the first two parts were produced by someone holding the secret and were not changed.

Rule: put only what the server needs to identify the caller. No balances, account numbers, names, emails or KYC data.

Who can sign, who can verify?



HS256 - shared secret (symmetric)

Used in this lab

- **One key** signs AND verifies
- **Simple:** one service issues and checks its own tokens
- **Risk:** every verifier can also mint tokens - share the key with nobody
- **Key:** ≥ 256 bits of randomness, from env / secrets manager



RS256 / ES256 - key pair (asymmetric)

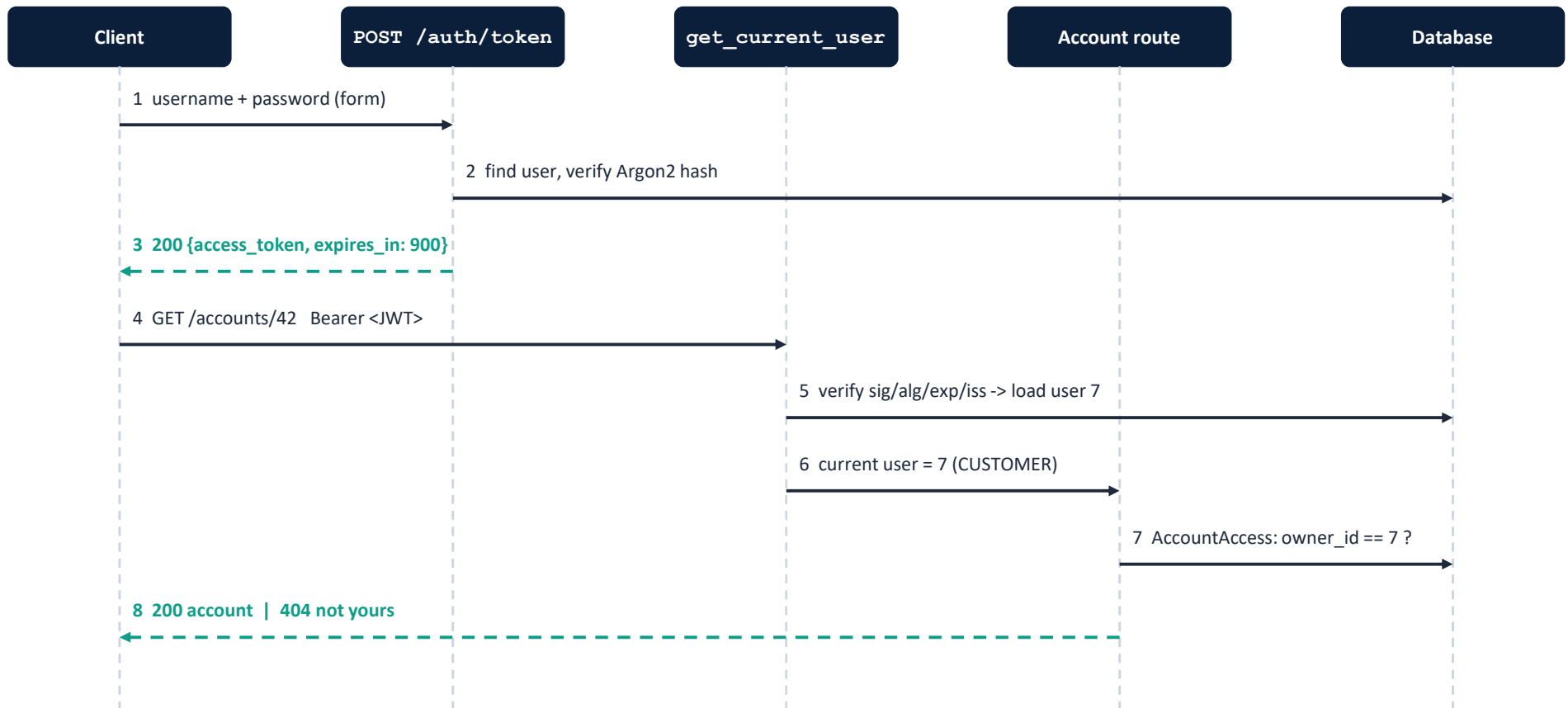
Typical in production

- **Private key** signs - only the auth server has it
- **Public key** verifies - published as a JWKS URL
- **Scales:** 20 microservices can verify without being able to forge
- **Rotation:** publish new key id (kid) before switching

```
# Never let the TOKEN choose the algorithm:  
jwt.decode(token, key, algorithms=["HS256"])  
# pinned list -> "alg": "none" and algorithm-confusion attacks fail
```

END-TO-END FLOW

The token lifecycle inside bank_api



What "validate the token" really means

Check	How	Attack it stops
1. Token present?	Missing header -> 401 + WWW-Authenticate: Bearer	Anonymous access
2. Signature valid?	HMAC with our secret matches	Tampered payload (role -> ADMIN)
3. Algorithm pinned?	algorithms=["HS256"]	"alg": "none", key confusion
4. Not expired?	exp in the future (PyJWT checks)	Replay of stolen old tokens
5. Right issuer?	iss == "bank_api"	Tokens minted for another system
6. Required claims?	exp, iat, sub, iss all present	Tokens that never expire
7. User still valid?	user exists AND is_active	Deleted / disabled users

Any failure -> the SAME 401 "Invalid or expired token". Never tell an attacker which check failed.

THINK - PAIR - SHARE (5 MINUTES)

Exercise 1: spot the security flaw

A

```
{"sub": "jane.doe", "account_number": "1234567890123456",  
  "balance": 84250.00, "role": "CUSTOMER", "exp": 1672531199}
```

B

```
jwt.decode(token, SECRET, algorithms=jwt.get_unverified_header(token)["alg"])
```

C

```
SECRET_KEY = "secret"           # in config.py, committed to git
```

D

```
if not user or not verify(pw, user.hash):  
    raise HTTPException(401, "Wrong password for " + username)
```

E

```
token = create_access_token(user.id, user.role)    # exp = 30 days
```

Exercise 1: answers

- | | | |
|----------|---|---|
| A | Sensitive data in the payload | Payload is only Base64Url-encoded - anyone with the token reads the account number and balance. Keep only sub, role, iss, iat, exp. |
| B | Algorithm chosen by the attacker | The token picks its own algorithm (even "none"). Pin it: <code>algorithms=["HS256"]</code> . |
| C | Weak, leaked secret | Guessable and now in git history forever. Use ≥ 256 -bit random key from env / secrets manager; rotate if leaked. |
| D | User enumeration | Message reveals the username exists. Return one generic "Incorrect username or password", and hash-check even for unknown users (timing). |
| E | Token lives too long | A stolen token works for a month and cannot be revoked. Use 15 min access tokens (+ refresh tokens for UX). |

Same ideas, three frameworks

Concept	FastAPI (this lab)	Spring Security (Java)	ASP.NET Core (C#)
Read bearer token	<code>OAuth2PasswordBearer</code>	<code>oauth2ResourceServer().jwt()</code>	<code>AddJwtBearer(...)</code>
Verify JWT	<code>jwt.decode (PyJWT)</code>	<code>JwtDecoder (Nimbus)</code>	<code>TokenValidationParameters</code>
Current user	<code>Depends (get_current_user)</code>	<code>@AuthenticationPrincipal</code>	<code>HttpContext.User</code>
Protect all routes	<code>APIRouter(dependencies=[...])</code>	<code>authorizeHttpRequests(...)</code>	<code>RequireAuthorization() / [Authorize]</code>
Role check	<code>Depends (require_role("ADMIN"))</code>	<code>@PreAuthorize("hasRole('ADMIN')")</code>	<code>[Authorize(Roles = "Admin")]</code>
Ownership check	<code>AccountAccess dependency</code>	<code>custom AuthorizationManager</code>	<code>resource-based IAuthorizationService</code>
Hash passwords	<code>pwdlib (Argon2id)</code>	<code>Argon2PasswordEncoder</code>	<code>PasswordHasher<TUser></code>

Learn the concept once; the framework only changes the spelling.

Code: hashing and issuing tokens

```

from pwdlib import PasswordHash
import jwt                                # PyJWT

_password_hash = PasswordHash.recommended() # Argon2id, salted

def hash_password(plain: str) -> str:
    return _password_hash.hash(plain)

def verify_password(plain: str, hashed: str) -> bool:
    return _password_hash.verify(plain, hashed)

def create_access_token(user_id: int, role: str) -> str:
    now = datetime.now(timezone.utc)
    ttl = timedelta(minutes=settings.ACCESS_TOKEN_EXPIRE_MINUTES)
    claims = {
        "sub": str(user_id),                # MUST be a string
        "role": role,
        "iss": settings.JWT_ISSUER,
        "iat": now,
        "exp": now + ttl,                    # short-lived
    }
    return jwt.encode(claims, settings.JWT_SECRET_KEY,
                      algorithm=settings.JWT_ALGORITHM)

```



Library update (vs. older material)

- **python-jose -> PyJWT:** FastAPI docs now use PyJWT; python-jose is poorly maintained.
- **passlib -> pwdlib:** passlib is unmaintained; pwdlib + Argon2 is the current recommendation.

Gotchas

- pip install PyJWT (NOT "jwt" - a different package)
- PyJWT ≥ 2.10 raises InvalidSubjectError if sub is an int
- Secret comes from settings, never a literal

Code: turning a token into a User

```

oauth2_scheme = OAuth2PasswordBearer(tokenUrl="/api/v1/auth/token",
                                       auto_error=False)

def get_current_user(token: str | None = Depends(oauth2_scheme),
                    users: UserRepository = Depends()) -> User:
    if not token:
        raise NotAuthenticatedError("Not authenticated")
    try:
        claims = jwt.decode(token, settings.JWT_SECRET_KEY,
                           algorithms=[settings.JWT_ALGORITHM],
                           issuer=settings.JWT_ISSUER,
                           options={"require":
                                   ["exp", "iat", "sub", "iss"]})
        user_id = int(claims["sub"])
    except (jwt.PyJWTError, ValueError, KeyError):
        raise NotAuthenticatedError("Invalid or expired token")

    user = users.get_by_id(user_id)          # stateless != blind
    if user is None or not user.is_active:
        raise NotAuthenticatedError("Invalid or expired token")
    return user

```



auto_error=False

We raise our own 401 so it uses the Extension 5 error envelope and code "not_authenticated".



One except, one message

Expired, forged, garbage - the client learns nothing about which check failed.



Re-load the user

One indexed lookup buys instant lock-out of disabled users and live role changes.

ROUTE LEVEL VS ROUTER LEVEL

Code: locking doors with dependencies

```
app/core/auth.py

def require_role(*allowed_roles: str):          # a dependency FACTORY
    def checker(user: User = Depends(get_current_user)) -> User:
        if user.role not in allowed_roles:
            raise ForbiddenError("Insufficient role for this operation")
        return user
    return checker
```

```
router level - AuthN everywhere

# ONE line protects every route on the router
router = APIRouter(
    prefix="/accounts",
    dependencies=[Depends(get_current_user)])
```

```
route level - AuthZ where needed

# ...then tighten single routes
@router.put(
   ("/{account_id}/kyc-status",
    dependencies=[Depends(require_role(Role.ADMIN))])
def set_kyc_status(...): ...
```



Secure by default: protect the whole router, then open up only what must be public (register, token). Forgetting a decorator on a new route then fails CLOSED, not open.

Roles are not enough: object-level authorization

```
# Asha (user 7): valid CUSTOMER, valid token
# Account 42 belongs to Ravi
GET /api/v1/accounts/42
Authorization: Bearer <Asha's token>

200 OK
{"owner_name": "Ravi", "balance": 84250.0} # breach!
```

Authentication passed. Role check passed. Still a breach: nobody asked "is account 42 YOURS?". Broken Object Level Authorization is the #1 API risk because every endpoint with an id in the path needs this check.

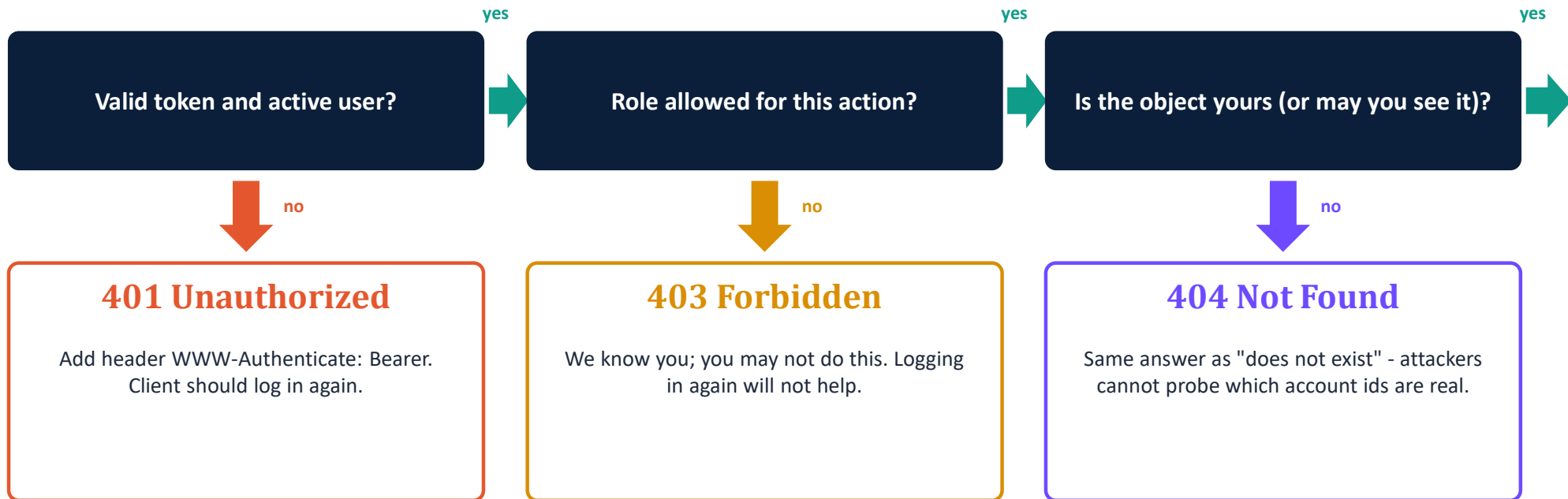
bank_api policy (what the lab enforces)

Action	Owner	Other customer	Admin
View account / history / schedules	Yes	404	Yes
Deposit / withdraw / transfer FROM	Yes	404	403
Transfer TO	Anyone	Anyone	-
Certify KYC	403	403	Yes
List / disable users	403	403	Yes

Least privilege: an admin can SEE customer accounts for support and compliance, but cannot move customer money. Different verbs, different rules.

STATUS CODES

401, 403 or 404? A decision you make on purpose



Exception worth noticing: an ADMIN trying to withdraw from a customer account gets 403, not 404 - admins can already see the account exists, so hiding it adds nothing. Consistency with your threat model beats a blanket rule.

Token lifetime, revocation and client storage



Keep access tokens short

- **15 min** in bank_api (ACCESS_TOKEN_EXPIRE_MINUTES)
- **Refresh token:** long-lived, stored server-side, rotated on every use
- **Logout** = revoke the refresh token



Revoking before exp

- **Disable the user:** works instantly because we re-load the user
- **Denylist by jti:** for "log out this one device"
- **Rotate the key:** nuclear option - logs everyone out



Where clients keep tokens

- **Web:** HttpOnly + Secure + SameSite cookie, or memory
- **Not localStorage:** any XSS bug can read it
- **Mobile:** iOS Keychain / Android Keystore

Refresh tokens and a jti denylist are STRETCH goals in the lab - the core tasks give you everything else on this slide.

BEST PRACTICES

Production checklist



Secret from env / vault

≥ 256 random bits; app refuses to start without it



Short access tokens

15-60 min; refresh tokens for UX



Argon2id / bcrypt

never plain or fast hashes



Server decides roles

reject unknown fields ("role": "ADMIN" -> 422)



HTTPS everywhere

bearer tokens are only as safe as the channel



Pin the algorithm

algorithms=["HS256"] (or RS256 with JWKS)



Minimal claims

sub, role, iss, iat, exp - no PII or money data



Generic login errors

same message + same timing for all failures



Check ownership

on EVERY route with an object id (BOLA)



Rate-limit login

slow down credential stuffing and brute force

Key takeaways



AuthN then AuthZ

Who are you -> what may you do. 401 vs 403 vs 404.



Hash, don't store

Argon2id with a salt. Follow NIST length rules.



JWT = signed, not secret

Readable by anyone. Pin alg, check exp and iss.



Roles AND ownership

RBAC alone leaves BOLA - OWASP API risk #1.



Dependencies are policy

get_current_user -> require_role -> AccountAccess.



Secure by default

Protect the router, open only what must be public.

YOUR MISSION • ~2.5 HOURS + STRETCH

Hands-on lab: bank_api Extension 6

1 Users & hashing

register, Argon2id, no self-made admins

`test_auth1_users.py`

2 Login -> JWT

POST /auth/token, minimal claims

`test_auth2_login.py`

3 Who is calling?

get_current_user, GET /auth/me

`test_auth3_current_user.py`

4 Lock the doors

every money route needs a token

`test_auth4_protected_routes.py`

5 Is it yours?

owner_id + AccountAccess (BOLA)

`test_auth5_ownership.py`

6 Roles

admin-only KYC, admin views, disable users

`test_auth6_roles.py`

Start: starter zip + Lab Guide **Done:** `pytest -v -> 108 passed`

Stuck? Ext6_Unstuck_Hints.pptx (hints before code)